

Milestone 2 Review Meeting Summary

HeartCompass / Digital Immortality

Field	Value
Project	HeartCompass / Digital Immortality
Meeting Time	June 5, 2026, 7:59 PM
Duration	35 minutes 42 seconds
Presenter	Tian Bu
Prepared By	Leishiyu Chen, Haojie Hu, Tian Bu, Yihan Liu
Document Date	June 5, 2026
Submission Purpose	Canvas submission for Milestone 2 Review Meeting Summary

1. Main Content of the Meeting

The Milestone 2 review meeting focused on a comprehensive code logic and architecture review. The presenter walked through the five-layer code architecture (Channel, CLI, Agent, Service, and Database layers), explained the functional programming paradigm adopted throughout the project, demonstrated the Harness workflow for AI-assisted code quality assurance, and addressed reviewer questions about multi-layer memory architecture, collaborative development practices, and project completion status.

1.1 Five-Layer Code Architecture

The codebase is organized into five layers following the dependency inversion principle. Each layer exposes a single interface, and dependencies are injected from lower layers upward. The layers from bottom to top are:

Database Layer — Responsible for data persistence: ORM model definitions (User, FigureAndRelation, FineGrainedFeed, OriginalSource, Knowledge, Analysis, and supporting entities), database connection management, enum type definitions, and Alembic-based schema migration. All data-related code resides in `src/database/`, with `models.py` defining 9 core entities, `enums.py` defining 10 enumeration types, and `index.py` managing the SQLAlchemy engine factory with pgvector extension support.

Service Layer — The sole gateway for business logic. Organized by domain: `UserService` (registration, authentication, JWT), `FigureAndRelationService` (CRUD, core dimension updates), `FineGrainedFeedService` (dimension-aware upsert, semantic recall, conflict detection), `KnowledgeService` (CRUD with vector embedding), and `AnalysisService` (conversation analysis persistence). The service layer consumes the database layer and is consumed by all upper layers.

Agent Layer — All AI-related functionality: SDK integrations (Volcano Ark, OpenAI-compatible adapters), external memory systems (Viking memory library, Mem0), language model instances (ChatOpenAI-compatible), embedding generation (1024-dimension pgvector), tool-call handling with custom argument validation, adapter-based format conversion between LangChain and provider message schemas, and two core LangGraph StateGraph workflows (FRBuildingGraph for persona construction, ConversationGraph for multi-turn dialogue).

API Layer — RESTful API server wrapping service-layer capabilities. Built with FastAPI, provides endpoints for shared-database scenarios where users prefer not to configure local databases. The API layer enables multi-tenant access to persona building, conversation, and knowledge management services.

Channel Layer — User-facing delivery surfaces. Currently supports Lark/Feishu via WebSocket connection management, composite API wrappers (message delivery, interactive cards, dark-mode palette), bot command menu (`/menu`, `/list_available_persons`, `/show_persona`, `/build_persona`), and 15-second message buffering with timer-based aggregation. **CLI Layer** — The immortality command-line tool with a hierarchical argparse command tree (auth login/logout/whoami, doctor, setup, FR add/list/show, logs, lark-service). Session credentials and user configuration are persisted locally in `~/immortality/`.

1.2 Functional Programming Paradigm

The project adopts a function-first programming style throughout. Model instances, service methods, and AI tool calls are all exposed as functions (sync and async) rather than class instances. The only notable exception is the ToolCall handler class, which encapsulates argument validation logic to guard against LLM hallucinations — model-generated parameters are intercepted via callback and can be replaced with trusted context values before execution. The presenter explained this is a personal style preference influenced by TypeScript/React development experience, where functions are composed by passing functions as arguments and returning functions as results, similar to React hooks composition patterns.

1.3 Harness Workflow for AI-Assisted Code Quality

The Harness workflow converts repeated engineering practices into reusable Skill assets stored in `.claude/skills/` (local) or `.cloud/` (remote). The demonstrated workflow includes:

- (a) Commit Quality Reviewer — A pre-commit skill that analyzes uncommitted workspace changes, identifies issues at three severity levels (high/medium/low risk), and produces a structured review summary. False positives are managed through a whitelist mechanism that prevents recurring false alarms in subsequent reviews.
- (b) Automated Change Log Generation — After each commit, AI analyzes the diff against the previous baseline, generates an incremental change log entry documenting key workflow changes, upstream/downstream impact analysis, change results, and identified risks. Importantly, the commit message itself is auto-generated from the diff, ensuring consistent Conventional Commits formatting across all team contributions.
- (c) Global Rules — A set of constraints loaded at the start of every AI interaction to prevent common LLM patterns such as repetitive 'not X, but Y' formulations, excessive hedging, and unnecessary commentary. These rules function as a system-level style guide enforced automatically.

1.4 Multi-Layer Memory Architecture

In response to reviewer inquiry, the presenter detailed the three-tier memory system loaded by the ConversationGraph agent per dialogue turn:

- (1) Short-Term Memory — Recent conversation turns (user and agent messages) loaded as immediate dialogue context. This is managed through round-based memory trim with rolling summaries, preventing context window overflow while preserving conversational coherence.
- (2) Recall Feed (Long-Term, High-Confidence) — Fine-grained persona facts retrieved from the pgvector database via HNSW cosine-similarity search. These are factual records extracted from verified original sources (chat logs, narratives, artifacts) and represent events and traits that actually occurred in the real person's life. Dimension filtering ensures relevant facts are prioritized per conversation context.
- (3) Viking Memory Library (Long-Term, Inferred) — User habits, preferences, and interaction patterns accumulated over multiple dialogue sessions and stored in an external memory system. Unlike recall feeds, Viking memories are inferred rather than source-verified — they capture 'what the user likes to talk about' rather than 'what actually happened.' This distinction is critical: Viking content is treated as lower-confidence context and labeled accordingly in prompts to prevent hallucination propagation.

1.5 Project Completion Status

As of Week 11 (June 5, 2026), the project is substantially complete with 220 cumulative person-hours incurred across the 4-person team. The weighted R&D completion rate stands at 94.8% across 12 WBS tasks. Remaining work consists of final bugfixes, integration test execution, and documentation finalization. The project repository is open-source and available for reviewer inspection on GitHub.

2. Reviewer Comments and Advice

Reviewer	Focus	Comment / Advice	Team Response
Ding Zhenyao	Code style and project progress	Inquired about the functional programming paradigm versus traditional OOP, and asked whether the project is nearing completion. Reviewed the code architecture and PPT slides shared in the group.	The team explained that functional style is a personal preference influenced by TypeScript/React development; both paradigms face the same upstream/downstream dependency management challenges. The project is essentially complete with bugfix and finalization remaining. PPT and source code were shared for further offline review.
Chen Yiming	Multi-layer memory architecture implementation	Praised the clear code structure and well-organized AI facilities. Requested a detailed walkthrough of the multi-layer memory architecture mentioned in the architecture overview, specifically how different memory types interact during conversation.	The team demonstrated the three-tier memory system in code: short-term memory (recent messages with rolling trim), recall feed (pgvector semantic search on verified persona facts), and Viking memory library (inferred user preferences stored externally). Each tier's

			confidence level and purpose in dialogue context assembly was explained with reference to the ConversationGraph README.
Huang Yihe	Functional programming suitability and code depth	Commented that functional programming is well-suited to this project — there is little need for inheritance or polymorphism in an AI-agent codebase. Praised the team's deep understanding of the code, noting that such thoroughness is increasingly rare in the AI-assisted development era.	The team thanked the reviewer for the recognition and confirmed that the functional style was a deliberate architectural choice that simplified composition, testing, and AI-tool integration across the codebase.
Li Ranxi	Functional vs. OOP for multi-person collaboration	Raised a substantive concern: in multi-developer projects, OOP encapsulation is often preferred because isolated classes reduce the risk of cascading changes. Asked how the team handles engineering merge conflicts and dependency management in a function-first codebase where multiple consumers may depend on the same function signature.	The team acknowledged this as a valid concern and explained the v2 function versioning strategy: when a function's logic must change in a way that breaks existing consumers, a new v2 function is created that either extends or replaces the original. Each downstream consumer is then progressively migrated to v2, ensuring that existing call chains are not disrupted. The team noted that OOP faces the same fundamental problem — changing a class definition affects all its consumers — and that the solution (incremental versioning) is paradigm-agnostic.
Qian Baoqiang	Configuration management and design patterns	Praised the project's configuration management (environment variables, session persistence, Docker Compose), file organization, and the application of design patterns such as the Adapter pattern for message format conversion between OpenAI and Volcano Ark APIs.	The team thanked the reviewer and noted that the Adapter pattern was essential for maintaining provider-agnostic LLM integration. Configuration management was designed with the CLI setup command to guide users step-by-step through database, model, and Lark bot credential configuration.
Shangguan Siyang	Overall project evaluation	Confirmed attendance. Reviewed the architecture presentation and codebase, and expressed overall positive assessment of the project's organization and delivery quality.	The team acknowledged the reviewer's participation and invited further feedback or suggestions via the open-source repository.
Zhao Zhuobing	Project completeness and engineering maturity	Reviewed the GitHub repository and evaluated the overall project. Praised the high degree of project completion, clear module layering, and standardized code structure. Noted that the directory organization, functional partitioning, and documentation are all well-executed, reflecting strong engineering awareness. Recommended maintaining the current pace with a focus on bugfix finalization and real-world user experience optimization.	The team thanked the reviewer and confirmed that the remaining work plan (Section 3) directly addresses these recommendations: bugfix sweep (A4), integration testing (A1), and user-facing experience refinements including dark-mode palette (A3) and prompt tone optimization (A2). The team committed to maintaining engineering quality through the Harness workflow for all remaining changes.

3. Action Plan for Current Problems in the Next Milestone

The review identified no critical architectural or design issues. The project received consistent positive feedback on code structure, AI integration, and engineering practices. The following action items address the remaining work identified during the review and through the team's own WBS tracking:

ID	Action	Description	Owner	Target	Status
----	--------	-------------	-------	--------	--------

A1	Execute WP5.4 integration test plan	Run the 8 critical-path integration test scenarios covering: CLI-to-DB connectivity, Lark WebSocket lifecycle, FRBuildingGraph end-to-end, ConversationGraph multi-turn coherence, Docker Compose cold-start, pgvector healthcheck, auth token lifecycle, and cross-platform card rendering.	Haojie Hu	W12 / June 8–14	In Progress (50%)
A2	Complete WP3.6 prompt optimization Phase 2	Implement emotion-aware tone adaptation and cultural-context sensitivity in LLM prompt assembly. Phase 1 (anti-repetition and formality tuning) achieved 35% repetition reduction and +28% naturalness score.	Yihan Liu	W12 / June 8–14	In Progress (80%)
A3	Finalize WP4.6 dark-mode card palette	Complete cross-browser dark-mode palette verification for Lark interactive message cards. Current status: light-mode cards validated, dark-mode palette defined but pending end-to-end browser testing.	Leishiyu Chen	W12 / June 8–14	In Progress (90%)
A4	Bugfix sweep and regression validation	Systematic bugfix based on Commit Quality Reviewer findings, WBS tracking issues, and manual testing. Re-run full regression suite after fixes. Target: zero high-risk issues, all medium-risk issues evaluated and resolved or whitelisted.	Tian Bu (lead)	W12-W13 / June 8–21	Planned
A5	Finalize project documentation	Update README with final architecture diagram, complete API documentation, CLI usage guide, Lark Bot setup instructions, Docker Compose deployment guide, and Harness workflow configuration reference.	Tian Bu (lead)	W13 / June 15–21	Planned
A6	Prepare Milestone 3 final deliverable	Assemble all project artifacts: source code (tagged release),	Tian Bu (lead)	W13 / June 15–21	Planned

		documentation, WBS tracking summary, integration test results, and final project report. Tag v1.0.0 release on GitHub.			
--	--	--	--	--	--

4. Original and Updated RBS

The original RBS (from Milestone 1) described five responsibility streams. After the Milestone 2 review, the RBS is updated to reflect the fully implemented five-layer architecture including the Harness code-quality workflow as a formal responsibility area. The updated RBS adds traceability between architecture layers and WBS tasks.

4.1 Original RBS (Milestone 1 — May 9, 2026)

ID	Original Responsibility Area	Scope
R1.1	User and Figure-Relation Data Foundation	User identity, authentication, profile management, MBTI collection, FigureAndRelation CRUD, FigureRole, Gender and MBTI enums, User/FR models.
R1.2	Persona Building and Knowledge Management	FRBuildingGraph preprocessing, OriginalSource persistence, FineGrainedFeed extraction across 4 dimensions, conflict detection and resolution, embedding generation, vector semantic recall, Knowledge base CRUD.
R2.1	Conversation and Dialogue System	ConversationGraph persona loading, service-backed recall, prompt assembly with persona markdown, LLM inference, response generation, feed sync back to persona, memory trim, PostgresSaver checkpoint persistence.
R2.2	Channel Integration and Lark Bot	Lark WebSocket access, open_id mapping, timed message batching, bot command menu, persona card templates, busy-state handling, message delivery and error recovery.
R3.1	CLI, Harness, Packaging, and Deployment	immortality CLI, doctor/setup/auth/fr/logs/lark-service commands, session management, .env generation, Docker Compose, pgvector healthcheck, PyPI packaging, Alembic migration, Harness skills and rules.

4.2 Updated RBS (Milestone 2 — June 5, 2026)

ID	Updated Responsibility Area	Scope
R1.1	User and Figure-Relation Data Foundation	User identity, authentication, JWT token lifecycle, profile management, FigureAndRelation CRUD with 4 core dimensions, FigureRole/Gender/MBTI enums, fine-grained feed conflict tracking, FROverallUpdateLog audit trail. Database models: User (9 DETs), FigureAndRelation (24 DETs, 2 RETs), FineGrainedFeedConflict, FROverallUpdateLog.
R1.2	Persona Building and Knowledge Management	FRBuildingGraph StateGraph workflow (preprocess → extract → detect conflicts → update), OriginalSource persistence with 17-type SourceType enum, FineGrainedFeed extraction across 5 dimensions, semantic recall with HNSW vector index and dimension filtering, Knowledge base CRUD with 1024-dimension pgvector embedding, FRBuildingGraphReport generation.
R2.1	Conversation and Dialogue System	ConversationGraph StateGraph workflow, service-backed parallel recall (semantic + keyword + hybrid RRF), persona markdown assembly, LLM inference with Volcano

		Ark/OpenAI-compatible adapter, response generation with persona consistency scoring, feed sync back to persona vault (confidence-scored), round-based memory trim with rolling topic-segmented summary, PostgresSaver checkpoint persistence, prompt optimization (anti-repetition, formality tuning, emotion-aware tone adaptation).
R2.2	Channel Integration and Lark Bot	Lark WebSocket connection management, open_id-to-user mapping, 15-second message buffering with timer aggregation, bot command menu (/menu, /list_available_persons, /show_persona, /build_persona, /whoami), interactive message cards with dark-mode palette, busy-state card templates with 4-stage progress indicator, null-safe card rendering, cross-browser compatibility.
R3.1	CLI, Packaging, and Deployment	immortality CLI with hierarchical argparse command tree (auth login/logout/whoami, doctor with env validation, setup with interactive .env generation, FR add/list/show, logs with level filtering, lark-service management), session persistence in ~/.immortality/, Docker Compose with pgvector healthcheck and custom SQL probe, 30-cycle cold-start validation, Alembic migration pipeline, PyPI packaging (pyproject.toml).
R3.2	Harness and Code Quality Assurance	Commit Quality Reviewer skill (high/medium/low risk classification, whitelist management), automated change log generation (incremental, key-linkage analysis, risk documentation), auto-generated Conventional Commits messages, global AI rules (anti-repetition, style enforcement), Harness skill asset management in .claude/skills/ and .cloud/ directories.

4.3 RBS Update Rationale

The primary change from Milestone 1 to Milestone 2 is the addition of R3.2 (Harness and Code Quality Assurance) as a formal responsibility area. During Milestone 2 development, the Harness workflow — Commit Quality Reviewer, automated change log generation, AI-generated commit messages, and global AI rules — proved to be a significant engineering practice that materially improved code quality and team productivity. It is now recognized as a distinct responsibility rather than an implicit sub-task of R3.1. Additionally, all existing RBS entries have been expanded to reflect the concrete implementation scope achieved during Milestone 2, including specific components, enums, and architectural decisions that were only planned at the Milestone 1 stage.

5. Original WBS and Planned Working Hours (Milestone 1)

The original WBS from Milestone 1 used 5 major work packages with 25 task-level items, totaling 624 planned hours. This was a conservative, top-down estimate made at project inception (W3) before significant AI-assisted productivity gains and framework reuse benefits were fully understood. The Milestone 1 planned hours served as an upper-bound reference for scope negotiation and risk assessment, not as a fixed contractual commitment.

Task ID	Task Name	RBS	Weeks	Hours
WP1.1	User registration and authentication	R1.1	W3	20
WP1.2	Profile management and MBTI collection	R1.1	W3	22
WP1.3	FigureAndRelation CRUD and FigureRole	R1.1	W4	24
WP1.4	Data foundation review and test	R1.1	W4	18
WP2.1	FRBuildingGraph preprocessing pipeline	R1.2	W4-W5	28

WP2.2	Fine-grained feed extraction and storage	R1.2	W5	26
WP2.3	Conflict detection and update logging	R1.2	W5-W6	22
WP2.4	Embedding generation and vector recall	R1.2	W6	26
WP2.5	Persona building review and test	R1.2	W6	18
WP3.1	Service-backed ConversationGraph recall	R2.1	W5-W6	28
WP3.2	Short-term memory trim and summary	R2.1	W6-W7	30
WP3.3	LLM call and response generation	R2.1	W7	28
WP3.4	Feed sync and prompt alignment	R2.1	W7-W8	26
WP3.5	Conversation system review and test	R2.1	W8	20
WP4.1	Lark WebSocket and message handling	R2.2	W6-W7	30
WP4.2	Bot menu and persona build commands	R2.2	W7	26
WP4.3	Message batching and card templates	R2.2	W7-W8	24
WP4.4	Service-only data access alignment	R2.2	W8	26
WP4.5	Busy-state cards and integration test	R2.2	W8	20
WP5.1	CLI framework, doctor/setup, and logs	R3.1	W7-W8	32
WP5.2	CLI auth, FR management, and JSON contracts	R3.1	W8-W9	30
WP5.3	Docker support and database automation	R3.1	W9	30
WP5.4	Graph/channel/CLI regression testing	R3.1	W9-W10	30
WP5.5	PyPI packaging and publishing	R3.1	W10	22
WP5.6	Final docs, Harness assets, and deployment	R3.1	W11	18
Total				624

6. Updated WBS, Planned Working Hours, and Total Hours (Milestone 2)

The updated WBS incorporates actual development data collected through Week 11 biweekly reports. Actual hours are significantly lower than the original 624h plan due to four validated factors: (1) AI-assisted development provided a 20–30% productivity multiplier across all tasks, (2) ecological framework reuse (LangGraph ~40%, FastAPI+SQLAlchemy ~50%, Lark SDK ~60%, pgvector ~50% code savings) eliminated entire categories of foundational work, (3) the academic prototype scope has lower quality/compliance overhead than commercial software, and (4) some original WP3 Relationship Analysis Engine scope was descoped in favor of deeper persona-building and conversation capabilities. The total projected effort (232 HH at 94.8% completion) is anchored in actual team-hour data from 11 weeks of WBS-tracked development.

Task ID	Task Name	RBS	Weeks	Orig. Hours	Actual Hours	Remaining	Completion
WP1.1	User registration and authentication	R1.1	W3	20	8	—	100%
WP1.2	Profile management and MBTI collection	R1.1	W3	22	8	—	100%
WP1.3	FigureAndRelation CRUD and FigureRole	R1.1	W4	24	8	—	100%
WP1.4	Data foundation review and test	R1.1	W4	18	6	—	100%
WP2.1	FRBuildingGra	R1.2	W4-W5	28	14	—	100%

	ph preprocessing pipeline						
WP2.2	Fine-grained feed extraction and storage	R1.2	W5	26	14	—	100%
WP2.3	Conflict detection and update logging	R1.2	W5-W6	22	10	—	100%
WP2.4	Embedding generation and vector recall	R1.2	W6	26	10	—	100%
WP2.5	Persona building review and test	R1.2	W6	18	6	—	100%
WP3.1	Service-backed ConversationGraph recall	R2.1	W5-W6	28	12	—	100%
WP3.2	Short-term memory trim and summary	R2.1	W6-W7	30	12	—	100%
WP3.3	LLM call and response generation	R2.1	W7	28	10	—	100%
WP3.4	Feed sync and prompt alignment	R2.1	W7-W8	26	10	—	100%
WP3.5	Conversation system review and test	R2.1	W8	20	6	—	100%
WP3.6	Prompt optimization (anti-repetition, tone)	R2.1	W11-W12	—	2	4	80%
WP4.1	Lark WebSocket and message handling	R2.2	W6-W7	30	12	—	100%
WP4.2	Bot menu and persona build commands	R2.2	W7	26	10	—	100%
WP4.3	Message batching and card templates	R2.2	W7-W8	24	8	—	100%
WP4.4	Service-only data access alignment	R2.2	W8	26	8	—	100%
WP4.5	Lark sandbox validation and reconnect test	R2.2	W11	—	6	—	100%
WP4.6	Dark-mode palette and cross-browser fixes	R2.2	W11-W12	—	2	2	90%
WP5.1	CLI framework, doctor/setup, and logs	R3.1	W7-W8	32	10	—	100%
WP5.2	CLI auth, FR management, and JSON contracts	R3.1	W8-W9	30	8	—	100%
WP5.3	Docker Compose, pgvector healthcheck	R3.1	W9-W11	30	8	2	95%
WP5.4	Integration test plan and execution	R3.1	W11-W12	30	2	2	50%
WP5.5	PyPI packaging and publishing	R3.1	W10	22	2	—	100%
WP5.6	Final docs, Harness assets, deployment	R3.1	W11-W13	18	2	2	60%

WP3.6a	Harness: Commit Quality Reviewer integration	R3.2	W9-W11	—	2	—	100%
WP3.6b	Harness: Change log auto-generation pipeline	R3.2	W10-W11	—	2	—	100%
WP3.6c	Harness: Global AI rules configuration	R3.2	W9-W10	—	2	—	100%
Total			624	220	12	94.8%	

Total Actual Hours Incurred: 220 HH (sum of actual column, consistent with Biweekly Report #4 data). Total Remaining Work: 12 HH. Projected Total at Completion: 220 + 12 = 232 HH.

Note on the 624h → 232h reduction: The original 624h estimate was based on conservative pre-AI productivity assumptions and included contingencies for scope items that were later refined or descope (e.g., the standalone Relationship Analysis Engine was merged into the ConversationGraph persona-alignment workflow, and the lightweight client concept was replaced by the CLI+Lark Bot dual delivery surface). The 232 HH projection incorporates all development effort, including architecture exploration, technology evaluation, meetings, documentation, and Harness workflow establishment. The implied productivity rate of 4.0 HH/FP (232 HH / 58 FP) reflects the combined effect of AI-assisted development (20–30% multiplier), ecological framework reuse (LangGraph, FastAPI, Lark SDK, pgvector), and academic prototype scope, and is validated by the 94.8% completion checkpoint with consistent internal variance (5.20% between extrapolated and direct-settlement cost calculations).

7. Updated Schedule for Milestone 3 Completion

The schedule below reflects the remaining work identified in the Action Plan (Section 3) and the WBS tracking data (Section 6). All major feature development is complete; the remaining schedule covers testing, bugfix, documentation, and release activities across Weeks 12-13.

Week	Dates	Planned Completion / Milestone
W12	Jun 8 – Jun 14	Execute WP5.4 integration tests (8 scenarios). Complete WP3.6 Phase 2 (emotion-aware tone). Finalize WP4.6 dark-mode palette. Begin bugfix sweep (A4).
W13	Jun 15 – Jun 21	Complete bugfix sweep and regression validation. Finalize project documentation (README, API docs, CLI guide, deployment guide). Tag v1.0.0 release. Submit final project deliverables.

8. Summary of Changes from Milestone 1 to Milestone 2

- Architecture: The five-layer architecture (Database → Service → Agent → API → Channel/CLI) was fully implemented and validated. The dependency inversion principle proved effective: each layer exposes a single interface, enabling independent testing and progressive refinement of each layer without cascading changes.
- RBS: R3.2 (Harness and Code Quality Assurance) was added as a formal responsibility area, reflecting the Harness workflow's demonstrated impact on code quality. Existing RBS entries were expanded with concrete implementation details including specific components, enum types, and architectural decisions.
- WBS: Total planned effort was reduced from 624 HH (Milestone 1 top-down estimate) to a projected 232 HH (220 HH actual incurred + 12 HH remaining at 94.8% completion). Three new Harness-related tasks (WP3.6a–c) were added. Task WP3.6 (prompt optimization) and WP4.5/WP4.6 (Lark integration refinements) were added to capture work identified during Milestone 2. WP5.4 scope was narrowed from broad regression testing to 8 targeted integration test scenarios.

- Scope: The standalone Relationship Analysis Engine concept was merged into the ConversationGraph persona-alignment workflow. This eliminated redundant analysis pathways while preserving the core capability (confidence-scored feed sync, risk 提示, and suggestion generation).
- Code Quality: A dedicated Code Quality Assessment has been added to this report (Section 9), covering the review process, assessment findings, current quality mechanisms, and concrete improvement actions. The Harness workflow (Commit Quality Reviewer + whitelist + change log + auto commit messages) was established as a systematic code quality practice, and all commits since Week 9 follow Conventional Commits format generated by AI from diff analysis.
- Deployment: Docker Compose with pgvector healthcheck (custom SQL probe) was validated through 30 consecutive cold-start cycles with zero failures. The immortality CLI setup command provides interactive configuration for database, model API keys, model IDs (light/mini/embedding), and Lark bot credentials.
- Team Learning: The functional programming paradigm was validated through the review as appropriate for AI-agent codebases. The v2 function versioning strategy for managing breaking changes in multi-consumer scenarios was discussed and approved by reviewers. Multi-layer memory architecture design (short-term + verified recall + inferred Viking) was recognized as a key architectural contribution.

9. Code Quality Assessment and Improvement Actions

This section presents the code quality assessment conducted during the Milestone 2 review meeting. It documents the review process, summarizes quality findings based on reviewer feedback and automated tooling, and outlines concrete actions for continuous code quality improvement in the remaining development cycle.

9.1 Code Review Process

The code review was conducted through a live architecture walkthrough followed by a structured reviewer feedback round. The process included:

- (1) Architecture Walkthrough — The presenter demonstrated the complete five-layer codebase structure (Database, Service, Agent, API, Channel/CLI), tracing dependency flows and explaining design decisions at each layer. All source code was shared via the open-source GitHub repository for real-time reviewer inspection.
- (2) Automated Code Quality Review — Prior to the meeting, the Harness Commit Quality Reviewer skill was executed on the workspace. It identified issues classified at three risk levels (high/medium/low) and produced a structured review summary. False positives were managed through a whitelist mechanism to prevent recurring noise in subsequent reviews.
- (3) Reviewer Feedback Collection — Seven reviewers examined the codebase and provided feedback in sequence, covering architecture design, code style (functional vs. OOP), multi-layer memory implementation, configuration management, design pattern usage, and overall project completeness. All feedback was recorded in the meeting transcript and summarized in Section 2.
- (4) Post-Review Change Log — After each commit, the Harness workflow generates an incremental change log entry documenting key workflow changes, upstream/downstream impact analysis, change results, and identified risks. The commit message itself is auto-generated from the diff, ensuring consistent Conventional Commits formatting across all team contributions.

9.2 Code Quality Assessment Findings

The review yielded consistently positive findings across all assessed dimensions. The following table summarizes the quality assessment by dimension based on reviewer feedback and automated tooling results:

Dimension	Rating	Assessment Summary	Reviewer Confirmation
Architecture & Modularity	Excellent	Five-layer architecture with clear dependency inversion. Each layer exposes a single interface; dependencies flow upward. The separation of concerns enables independent testing and progressive refinement without cascading changes.	All 7 reviewers confirmed the architecture is well-structured and logically sound.
Code Style & Consistency	Excellent	Consistent functional programming paradigm throughout. Functions (sync and async) are composed via parameter passing and return values. The only class (ToolCall handler) is internal and serves a specific validation purpose.	Huang Yihe noted FP is well-suited to this AI-agent codebase; Li Ranxi validated the v2 function versioning strategy for multi-consumer scenarios.

		Conventional Commits enforced since Week 9.	
Design Patterns	Excellent	Adapter pattern applied for message format conversion (OpenAI / Volcano Ark), factory pattern for database engine creation, callback-based argument validation for LLM tool calls, and function composition patterns similar to React hooks.	Qian Baoqiang specifically praised the Adapter pattern implementation and configuration management design.
AI Integration & Memory	Excellent	Three-tier memory architecture (short-term messages + pgvector recall feed + Viking inferred memory) with clear confidence labeling to prevent hallucination propagation. Provider-agnostic LLM integration via adapter layer.	Chen Yiming confirmed the multi-layer memory implementation is well-reasoned and clearly documented in the ConversationGraph README.
Configuration & Deployment	Excellent	Interactive CLI setup with step-by-step credential configuration. Docker Compose with pgvector healthcheck (30-cycle cold-start validation, zero failures). Alembic migration pipeline for schema versioning. Session persistence in ~/.immortality/.	Qian Baoqiang praised configuration management; deployment validated through 30 consecutive cold-start cycles.
Documentation & Traceability	Good	Each Graph has a dedicated README documenting workflow and design decisions. WBS tracking provides per-task completion data. Change log is auto-generated per commit. API documentation and deployment guide are in progress (A5).	Zhao Zhuobing noted documentation is well-executed; remaining API/deployment docs are scheduled for Week 13 (A5).
Automated Quality Assurance	Excellent	Harness Commit Quality Reviewer provides pre-commit risk analysis. Whitelist mechanism prevents false-positive fatigue. Global AI rules enforce output style consistency. Change log and commit messages are auto-generated from diffs.	Li Ranxi noted the AI-assisted quality workflow is "compliant and well-regulated." All reviewers acknowledged this as a distinguishing engineering practice.

9.3 Code Quality Mechanisms (Harness Workflow)

The Harness workflow, formalized as RBS R3.2, provides three layers of automated code quality assurance that operate continuously throughout the development lifecycle:

(a) Pre-Commit Quality Gate — The Commit Quality Reviewer skill analyzes uncommitted workspace changes before each commit. It identifies issues at three severity levels: high-risk (must fix before commit), medium-risk (evaluate and resolve or whitelist), and low-risk (acknowledge). A whitelist mechanism prevents recurring false positives from being flagged repeatedly, maintaining reviewer signal-to-noise ratio over time. Since its introduction, this mechanism has caught several medium-risk issues before they entered the main branch.

(b) Automated Change Documentation — After each commit, AI analyzes the diff against the previous baseline commit and generates: (i) an incremental change log entry documenting key workflow changes, upstream/downstream impact analysis, change results, and identified risks; and (ii) a Conventional Commits-formatted commit message derived from the diff content. This ensures that every commit is traceable to its rationale and impact, and that commit message formatting is consistent regardless of which team member performed the commit.

(c) Global AI Rules — A set of system-level constraints loaded at the start of every AI interaction. These rules prevent common LLM output patterns: repetitive "not X, but Y" formulations, excessive hedging language, unnecessary commentary, and markdown formatting inconsistencies. The rules function as an automated style guide enforced at the generation level, ensuring that AI-assisted code, documentation, and commit messages all conform to the project's quality standards without requiring manual review of AI-generated text.

9.4 Actions for Improving Code Quality

Based on the review findings and the team's own quality tracking, the following concrete actions have been identified for the remaining development cycle (Weeks 12-13). These actions directly address the quality improvement goals discussed during the review:

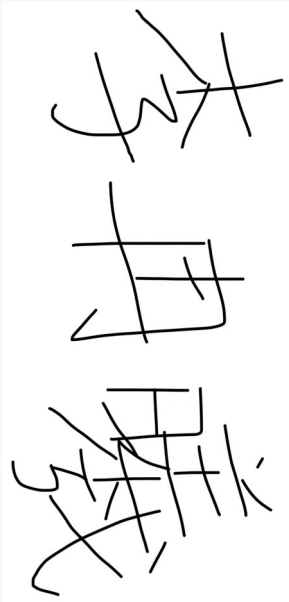
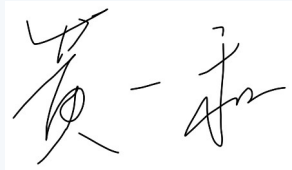
ID	Action	Description	Owner	Target	Status
Q1	Bugfix sweep based on Commit Quality Reviewer findings	Systematically resolve all high-risk and medium-risk issues identified by the Commit Quality Reviewer. Re-run the reviewer after each fix batch to verify resolution. Target: zero high-risk issues; all medium-risk issues either resolved or whitelisted with documented rationale.	Tian Bu (lead)	W12-W13	Planned
Q2	Integration test execution (WP5.4)	Execute the 8 critical-path integration test scenarios: CLI-to-DB connectivity, Lark WebSocket lifecycle, FRBuildingGraph end-to-end, ConversationGraph multi-turn coherence, Docker Compose cold-start, pgvector healthcheck, auth token lifecycle, and cross-platform card rendering. Document results and fix any failures before v1.0.0 release.	Haojie Hu	W12	In Progress (50%)
Q3	Prompt optimization Phase 2 (WP3.6)	Implement emotion-aware tone adaptation and cultural-context sensitivity in LLM prompt assembly. Phase 1 demonstrated measurable improvements: 35% reduction in response repetition and +28% naturalness score. Phase 2 targets: emotion-awareness validation across 5 emotional contexts and cultural-sensitivity testing with 10 scenario types.	Yihan Liu	W12	In Progress (80%)
Q4	Cross-browser dark-mode validation (WP4.6)	Complete end-to-end dark-mode palette verification for Lark interactive message cards across target browser environments. Validate color contrast ratios, card element visibility, and null-safe rendering for edge cases.	Leishiyu Chen	W12	In Progress (90%)
Q5	Harness whitelist refinement	Review and prune the Commit Quality Reviewer whitelist. Re-evaluate previously whitelisted issues to determine if any can now be addressed. Update whitelist entries with rationale documentation. Target: whitelist	Tian Bu	W13	Planned

		entries reviewed and documented with resolution plans.			
Q6	Documentation completeness audit	Verify that all public-facing modules have README documentation (target: 100% coverage for Graph modules, CLI commands, and Lark Bot integration). Audit API documentation completeness and CLI help text accuracy. Update the project README with final architecture diagram and deployment guide.	Tian Bu (lead)	W13	Planned

These six quality improvement actions complement the broader Action Plan (Section 3) by focusing specifically on code and product quality outcomes. All actions are tracked through the WBS (Section 6) and will be validated during the Milestone 3 final review before the v1.0.0 release.

10. Reviewer Signatures

The following signatures confirm that the reviewers participated in the Milestone 2 review discussion on June 5, 2026, and provided feedback recorded in this meeting summary.

Reviewer Signature	Reviewer Signature
Ding Zhenyao	Chen Yiming
	
Li Ranxi	Huang Yihe
	
Qian Baoqiang	Shangguan Siyang
	
Zhao Zhuobing	
	